

Data Mining

Block 2 - Regression, re-framed: from inference to prediction

Daniel Wlazło

SGH Warsaw School of Economics - SMMD-ADA

13 October 2026

Today, hour by hour

- | | | |
|---|---|---------|
| ① | Recap - what Block 1 left on the table | ~5 min |
| ② | Why these models - in the age of LLMs | ~10 min |
| ③ | Tools - scikit-learn; train / validation / test | ~10 min |
| ④ | From inference to prediction - overfitting, the split protocol | ~15 min |
| ⑤ | Linear regression - as a prediction machine | ~30 min |
| ⑥ | Logistic regression - the credit-scoring workhorse | ~35 min |
| ⑦ | Regularisation - ridge, lasso, and why scaling matters | ~20 min |
| ⑧ | Evaluating classifiers, part 1 - beyond accuracy | ~20 min |
| ⑨ | Lab - notebook 02_regression_baselines | ~35 min |

Breaks roughly every 50 minutes.

Last week in five sentences

- 1 Data mining starts from a **decision**; translating it into a learning task is your job.
- 2 **EDA before models**: target balance, distributions, bivariate reads - and always plot.
- 3 Missingness has a **mechanism**; the mechanism dictates the treatment.
- 4 The `delinq_missing` **indicator** we engineered carries more default signal than any raw numeric column.
- 5 Every feature must pass the **time-travel test** - application-time information only.

Today the craft graduates to a **real** portfolio - and we watch which lessons transfer. (Spoiler: engineered delay counters become the new star.)

Where we are on the map

Task	Target	Today's example
Regression	a number	payments from bills (warm-up)
Binary classification	two classes	probability of default

- Both of today's tools are *linear models* - one line of mathematics apart.
- They are also the **baselines** every fancier model of this course must beat to justify itself.

Meet the real portfolio

From today on, the modelling blocks run on **real data**:

- **30,000 credit-card clients** of a Taiwanese bank (2005) - the classic UCI *Default of Credit Card Clients* study (Yeh & Lien). Target: default next month - base rate **22.1%**.
- Columns: credit limit, demographics (sex, age, education, marriage), and **six months of behaviour** - repayment status, bill amounts, payment amounts.
- Loaded via `finance_data.load_taiwan()` - downloaded once, cached in the repo, works offline.
- One column is **planted and documented**: `collections_contact`, injected exactly like in Block 1's synthetic portfolio. You know the drill - and Block 3 pulls the pin.

Block 1's synthetic portfolio retires to the missingness lab - real data never labels its missingness mechanisms; ours did, by design.

Why bother, in the age of LLMs?

A fair question: ChatGPT exists - why learn logistic regression?

- **Structured data is not language.** On tabular problems (a credit portfolio is one), small dedicated models still **match or beat** LLMs - at a millionth of the cost per decision.
- **Explainability is a legal requirement.** A declined customer is entitled to reasons; “the LLM said so” is not one. Coefficients and odds ratios are.
- **Regulation.** Credit models live under model-risk frameworks: validated, monitored, auditable, reproducible. A deterministic model with fixed coefficients can be audited; a prompt cannot.
- **Insight.** The point is often the *drivers*, not the prediction: which behaviours signal risk? A coefficient answers; a chat completion gestures.
- **Latency, cost, stability.** Millions of scoring calls per day, identical answer for identical input, fractions of a cent.

Where LLMs *do* fit in this course's world

- **As your assistant:** writing pandas code, explaining errors, drafting documentation - use them for the lab, and **verify what they produce** (Block 1: reading errors).
- **As feature factories:** turning complaint texts into categories or embeddings that feed a *classical* model - exactly Block 5's text-mining pipeline, modern edition.
- **Not as the decision engine** for regulated, high-volume, structured decisions - for the reasons on the previous slide.

The division of labour

LLMs help you *build* the model. The model makes the decision. You remain responsible for both.

scikit-learn: the course's workhorse

- The standard Python library for classical machine learning - open source, two decades old, industry default for tabular work.
- One design idea worth the fame: **every model is an estimator with the same interface.**

Method	What it does	Exists on
<code>fit(X, y)</code>	learn parameters from training data	every estimator
<code>predict(X)</code>	output: a number or a class	every estimator
<code>predict_proba(X)</code>	output: class probabilities	classifiers
<code>transform(X)</code>	reshape features (scalers, encoders)	transformers

- Consequence: swapping logistic regression for a random forest next week is a **one-line change** - which is why baselines cost nothing to keep.

The API in six lines

One estimator, three verbs:

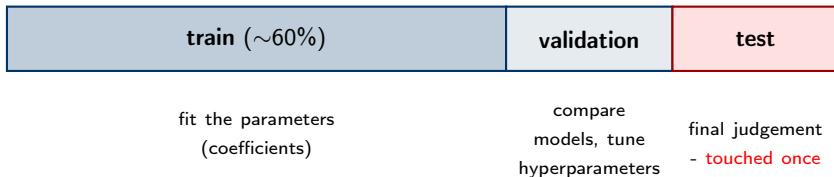
```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)                # learn from training data

y_class = model.predict(X_test)            # hard 0/1 decisions
y_proba = model.predict_proba(X_test)[:, 1] # P(default) per customer
```

- `predict` silently applies a 0.5 threshold - in credit we almost always want `predict_proba` and **our own threshold** (you'll see why after the break).
- The same four verbs run every model in Blocks 2–6.

Three samples, three jobs



- Three questions, three datasets: *what are the coefficients?* / *which model or setting?* / *how good is the winner, really?*
- Reusing one sample for two jobs is how optimism sneaks in - today's lab uses train + test only (no tuning yet); next week **cross-validation** makes the validation job cheap.

Same tool, new question

Econometrics asks

- is β_1 *significant*? what is its causal story?
- standard errors, confidence intervals, tests
- the model explains the *world*

Prediction asks

- how accurate is $\hat{y}$ on **customers we have never seen**?
- error on held-out data, stability over time
- the model supports a *decision*

Same $\hat{\beta}$'s, different loyalty. This course sits on the right - but borrows the left's discipline about interpretation.

On our portfolio: the econometrician asks “*does a payment delay cause default?*”; we ask “*is $\hat{P}(\text{default})$ accurate for tomorrow's applicants?*” - and both should ask “*can I defend this model to a regulator?*”

Generalisation: the only score that counts

- A model that perfectly reproduces its training data has proven one thing only: it can **memorise**.
- The customer whose application arrives tomorrow is not in the training set. Performance on *unseen* data - generalisation - is the product we sell.
- Everything in the next hour follows from one principle:

The golden rule of prediction

Never judge a model on the data it was fitted on.

(Beginner trap #1 from last week - now we build the defence.)

The train/test split

The protocol

- 1 Split once: e.g. 70% **train**, 30% **test**.
- 2 Fit the model (and every transform) on train only.
- 3 Judge on test - and touch the test set **as rarely as possible**.

Details that bite

- **Stratify** classification splits: keep the default rate equal in both halves.
- **Fix the seed** (`random_state`) - reproducibility, Block 1 style.
- Duplicates across the split = leakage (remember Block 1's keys check).
- With time-stamped data, split by *time*, not at random - more in Block 3.
- 70/30 with no validation part? Fine **today**: we tune nothing. The moment hyperparameters appear, carve validation out of the 70 - or cross-validate (Block 3).

The split, in code

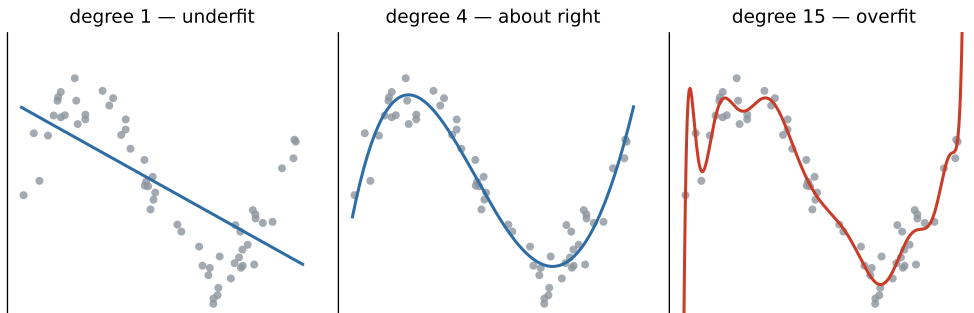
One import, one call - and two habits baked in:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    stratify=y,          # same default rate in both halves
    random_state=42,     # reproducible split
)
```

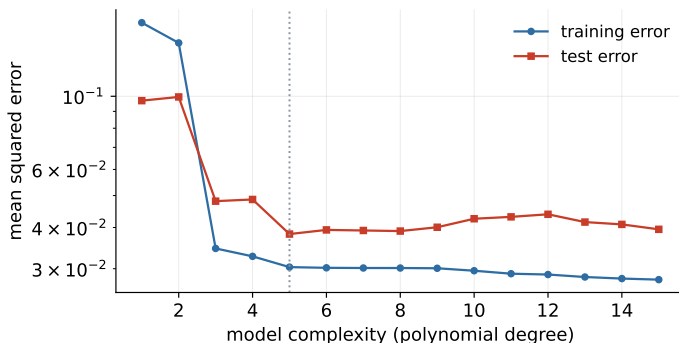
- From here on, `X_test`, `y_test` are locked in a drawer until judgement day.
- Every “how good is it?” during development must be answered *without* them - that is what validation and (next week) cross-validation are for.

What overfitting looks like



- Same data, three models. Degree 1 misses the structure (**underfit**); degree 15 memorises the noise (**overfit**).
- The overfit curve fits the training points *best* - that is exactly why training error cannot be trusted.

The U-curve every model rides



- Training error only goes down with complexity; **test error is a U** - underfitting on the left, overfitting on the right.
- Model selection = finding the bottom of the U *without looking at the test set*. Regularisation (later today) is one steering wheel.

Two more ways to fool yourself

Extrapolation

- A model is trustworthy *inside* the range of its training data.
- Our portfolio: credit limits up to 1M NT\$. A 5M NT\$ applicant? The model will output a number - **confidently and baselessly**.
- (Selection bias from Block 1 makes this worse: approved customers only.)

Test-set peeking

- Check the test score, tweak the model, check again, tweak again. . .
- Congratulations: you have manually fitted the test set. Its verdict is now **contaminated**.
- Rule of thumb: the test set answers *one* question, once, at the end.

The model

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

- y - numeric target (today's warm-up: **log payment**); x_j - features; ε - what the features cannot explain.
- **Linear in the coefficients**, not in the world: x_j can be $\log(\text{bill})$, an interaction flag, a squared term - feature engineering supplies the nonlinearity.
- Classical assumptions (independence, equal error variance, linearity) - we treat them as **things to check with plots**, not articles of faith. Residuals in three slides.

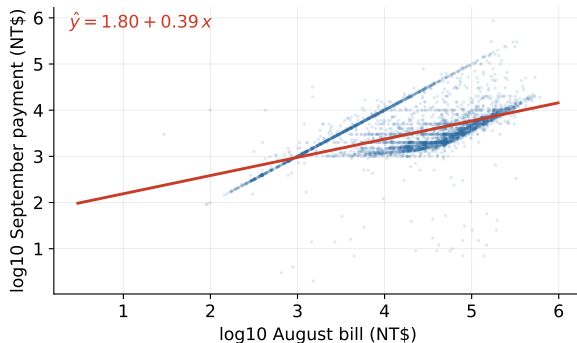
Least squares

Choose the coefficients that minimise the sum of squared mistakes:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_k x_{ik}$$

- Why squares? Big misses hurt disproportionately, the objective is smooth, and the solution is a closed formula - no iteration needed.
- The price: squares make OLS **sensitive to outliers** - one extreme customer can tilt the whole line (Block 1's boxplots just became modelling advice).
- Alternatives exist (absolute error, quantile regression) - good exam trivia, occasional real need.

Our first fit: payments vs bills



- How much of last month's bill shows up in this month's payment? Log-log (Block 1's lens): the slope is an **elasticity** - a 1% larger bill draws only $\sim 0.39\%$ more payment.
- Economics in one number: **revolvers absorb the rest** - bigger bills are financed, not repaid. The cloud is wide; let's quantify how wide.

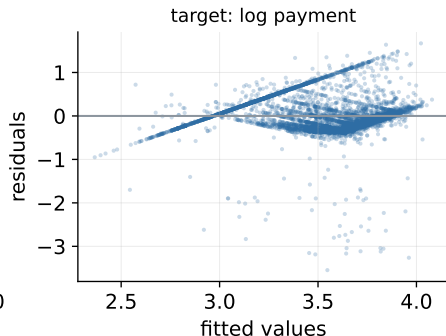
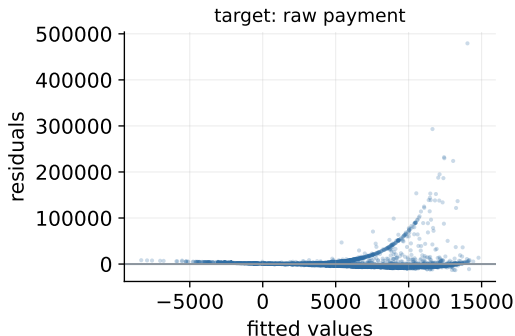
How good is the line? R^2 , RMSE, MAE

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2} \quad \text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

- R^2 : share of variance explained - unitless, comparable across targets. Our payment fit: $R^2 \approx 0.33$ on 24,605 paying customers.
- RMSE/MAE: in the *units of the target* - the language for business ("we miss by X NT\$ on average"). RMSE punishes big misses; MAE is robust.
- **Low R^2 is not failure**: real behavioural data is noisy, and a weak-but-real relationship can still price risk. The reverse matters more:

A **suspiciously high** R^2 on behavioural data means leakage or a target twin - investigate before celebrating (Block 1, beginner traps).

Residuals tell the truth



- Residual = $y - \hat{y}$, plotted against the fitted value. You want a **structureless band around zero**.
- Left (raw payments): a funnel - errors grow with the prediction (heteroscedasticity). Right (log payments): calm. *The transform fixed the model, not just the histogram.*
- Curvature in this plot $\Rightarrow$ missing nonlinearity $\Rightarrow$ engineer a feature.

Reading coefficients without embarrassment

- β_j = expected change in y per **one unit** of x_j , *holding the other features fixed* - units matter (per NT\$? per thousand? per SD?).
- To compare importance across features, **standardise** first (mean 0, SD 1): then β_j = effect of one SD.
- Coefficients are **teammates, not soloists**: adding or removing a correlated feature reshuffles the others (Block 1's correlation map warned you which ones).
- “Holding fixed” is a statistical statement, not a causal one - the Simpson's-paradox caution applies to coefficients too.

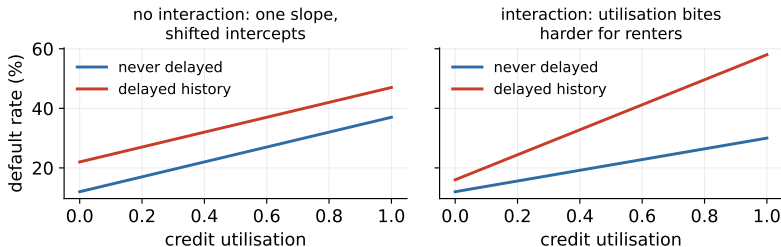
Categorical features enter the model: one-hot, carefully

- One-hot from Block 1: marriage → marriage_married, marriage_single, marriage_other.
- **Drop one level** (the *reference*): with an intercept, all levels together are perfectly collinear - the **dummy variable trap**.
- Each coefficient then reads “*versus the reference*”: β_{single} = single vs married, all else fixed.
- Choose the reference deliberately (largest or most natural level) - it changes the story, not the fit.

Ordinal features (checking status): integer codes keep the order - but assert equal gaps. Your call, stated out loud.

Interactions: when one effect depends on another

Illustrative data



$$y = \beta_0 + \beta_1 \text{util} + \beta_2 \text{delayed} + \beta_3 (\text{util} \times \text{delayed})$$

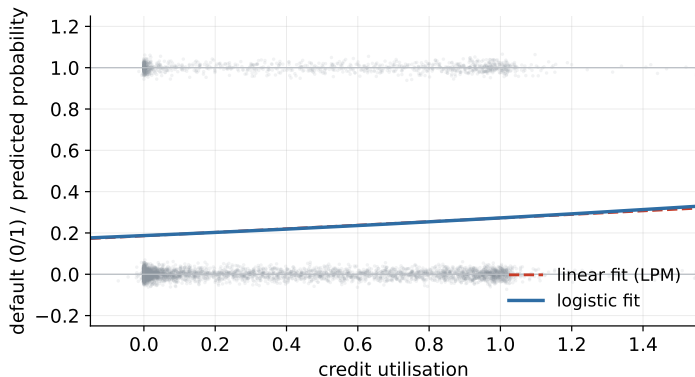
- With β_3 : the slope of utilisation becomes $\beta_1 + \beta_3 \text{delayed}$ - **one feature moderates another**.
- A linear model cannot discover this on its own: the product column is *feature engineering* (Block 1's interaction flag, graduated to a term). Trees find interactions automatically - next week.

Multicollinearity: correlated teammates

- When two features carry near-identical information (recall $|r| > 0.8$ pairs from the Block 1 heatmap), OLS cannot decide how to split the credit between them.
- Symptoms: coefficients with **absurd signs or sizes** that swing wildly between samples - while predictions stay fine.
- Consequence: *prediction survives, interpretation dies*.
- Remedies: drop one twin, combine them (a ratio!), or **regularise** - the third act of today.

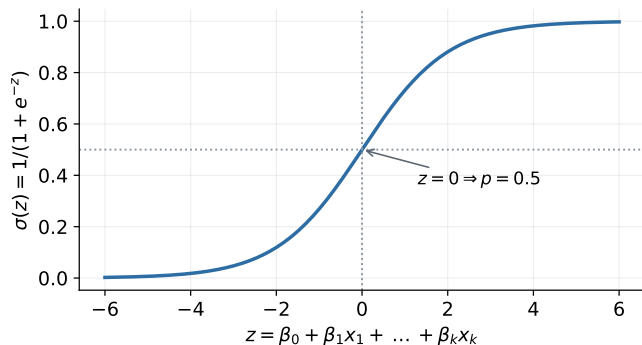
Diagnostic keyword for the curious: variance inflation factor (VIF).

When the target is 0/1, the line breaks



- Fit a straight line to default: predictions escape $[0, 1]$ - “-7% probability” and “130% probability” are not probabilities.
- We need a curve that *lives* in $[0, 1]$ and speaks probability natively. Enter the sigmoid.

From line to S-curve



- Keep the linear machinery $z = \beta_0 + \beta_1 x_1 + \dots$, then squash it: $p = \sigma(z) = 1/(1 + e^{-z})$.
- Any score in $(-\infty, \infty)$ becomes a probability in $(0, 1)$; the linear part still does all the thinking.

Odds and log-odds: the natural scale

$$\text{odds} = \frac{p}{1-p} \qquad \underbrace{\ln \frac{p}{1-p}}_{\text{log-odds (logit)}} = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

probability p	odds	log-odds
0.10	1:9 (0.11)	-2.20
0.50	1:1 (1.00)	0.00
0.75	3:1 (3.00)	+1.10
0.90	9:1 (9.00)	+2.20

- The model is **linear in log-odds** - that is the sense in which logistic regression is a linear model.
- Bookmakers have spoken odds for centuries; credit scoring inherited the dialect.

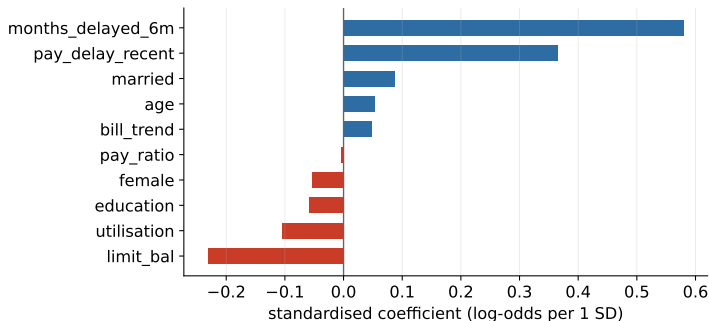
Fitting: maximum likelihood, log-loss

No sum of squares this time - we choose β to make the observed outcomes most probable:

$$\min_{\beta} -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i) \right] \quad (\text{log-loss / cross-entropy})$$

- Confidently wrong is punished brutally: predicting $\hat{p} = 0.99$ for a customer who repays costs $-\ln(0.01) \approx 4.6$; a humble 0.6 costs 0.9.
- No closed form - fitted iteratively. `sklearn` does this silently; you'll meet the same loss inside neural networks (Block 6).
- Same `fit/predict` API; plus `predict_proba` for the probabilities themselves.

Our PD baseline: what the model learned



- **Payment history is king:** the engineered delay counter `months_delayed_6m` (+0.58) and the most recent status (+0.37) tower over everything - Block 1's feature-engineering craft, vindicated on real data.
- And a haunting: `limit_bal` at -0.23 - higher limits, *lower* risk. The bank's **old screening** granted big limits to safe clients: selection bias (Block 1), visible inside a coefficient.

Odds ratios: the language of credit

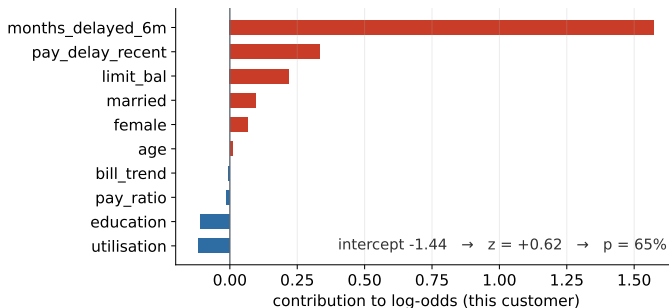
Exponentiate a coefficient and it becomes a **multiplier of odds**:

$$\text{odds}(x_j + \Delta) = e^{\beta_j \Delta} \cdot \text{odds}(x_j)$$

- Months delayed, +1 SD: odds of default $\times e^{0.58} \approx 1.8$.
- Credit limit, +1 SD: odds $\times e^{-0.23} \approx 0.79$ - a fifth of the odds gone (the screening echo, in odds language).
- Effects are **multiplicative**: two +1 SD delay-moves compound to $1.8 \times 1.8 \approx 3.2$, not +2×something.

“The odds double per X points” is exactly how scorecards talk - next slide.

Interpreting the model I: one customer, step by step



- Each bar = $\beta_j \times$ (this customer's standardised x_j): a full **receipt** for one prediction - contributions sum, with the intercept, to z , and $\sigma(z)$ is the PD.
- Adverse-action reasons fall straight out: the largest red bars. For logistic regression this decomposition is **exact and free** - SHAP (Block 3) exists to approximate it for models that aren't this polite.

Interpreting the model II: same odds ratio, different probability move

One SD more utilisation multiplies odds by $e^{0.43} \approx 1.54$ - *for everyone*. On the probability scale, it does not move everyone equally:

PD before	PD after (odds $\times$ 1.54)	change
10%	14.6%	+4.6 pp
50%	60.6%	+10.6 pp
90%	93.3%	+3.3 pp

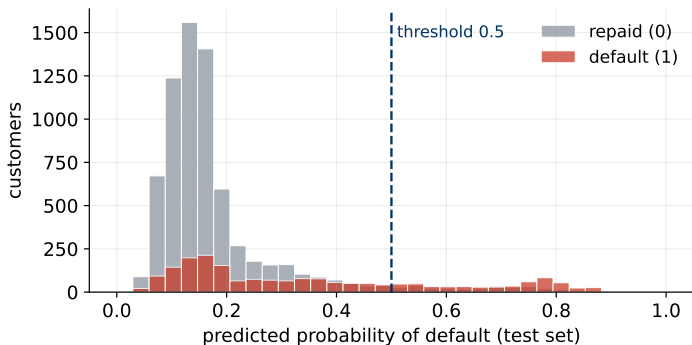
- Near the tails the sigmoid is flat; near $p = 0.5$ it is steepest - marginal effect $\partial p / \partial x_j = \beta_j p(1 - p)$.
- So: **odds ratios are constant, probability effects are not**. Always say which scale you are on - and report probability changes in *percentage points*, never “percent”.

Scores, not probabilities: the scorecard tradition

- A credit score is a **rescaled log-odds**: pick a base score, pick “points to double the odds” (say 20), done. Linear in the same z our model computes.
- Why banks kept it: additive per-feature points (from WoE-encoded bins, Block 1’s encodings slide) are auditable line by line - each feature’s contribution is a number a regulator can read.
- Logistic regression + WoE + scorecard formatting *is* the classical credit-scoring stack - decades old, still in production everywhere.

Modern practice: GBDT for power (next week), logistic scorecard for explainability - often both, side by side.

From probability to decision



- The two classes **overlap** - no threshold separates them cleanly; every cut-off trades one error for another.
- 0.5 is not a law of nature; it is `sklearn`'s default. The right threshold comes from *costs* - end of this block.

Class imbalance: three honest notes

- 1 Our 22% default rate is friendly; loan-book PD data (1–5%) pushes predicted probabilities low - at threshold 0.5 such a model may **never** predict “default”. That is a threshold problem, not a model problem.
- 2 `class_weight="balanced"` re-weights the loss - useful, but it **distorts predicted probabilities**; recalibration needed if probabilities matter (they do, in pricing).
- 3 Resampling (over/under-sampling) is popular and overrated - try thresholds and weights first.

Calibration - “is a predicted 0.3 really a 0.3?” - gets proper treatment in Block 3.

Why logistic regression is *the* baseline

- **Fast:** fits in milliseconds, scales to millions of rows.
- **Stable:** no tuning required to be decent.
- **Explainable:** coefficients $\rightarrow$ odds ratios $\rightarrow$ adverse-action reasons; regulators approve.
- **Honest yardstick:** any complex model must **earn its complexity** by beating this - measurably, on held-out data.

The baseline discipline

No GBDT, no neural network, no ensemble gets built in this course - or in your projects - before a logistic baseline exists.

Linear vs logistic, side by side

	Linear regression	Logistic regression
Target	numeric	binary (0/1)
Output	a number, unbounded	probability in (0, 1)
Linear in	the target	the <i>log-odds</i>
Fitted by	least squares (closed form)	max. likelihood (iterative)
Loss	squared error	log-loss
Read effects as	units of y per unit x	odds ratio e^{β}
Credit use	LGD, exposure, income est.	PD - the core task

One family, two dialects. Master both, and Block 3's models are "just" better function shapes.

Why shrink coefficients on purpose?

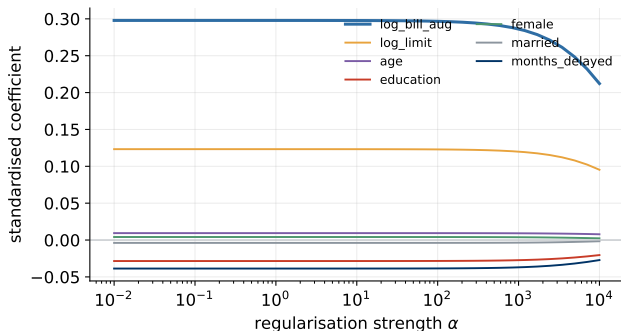
- Many features + correlated features + finite data $\Rightarrow$ coefficients tailored to **this sample's noise** - the overfitting U-curve, wearing a suit.
- Idea: add a **penalty for large coefficients** to the loss. The model must now buy each unit of coefficient with genuine predictive payoff.
- Price and prize: a little bias in, a lot of variance out - usually a bargain on test error.

Two classic penalties, two personalities: ridge and lasso.

Ridge (L2): smooth shrinkage

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j \beta_j^2$$

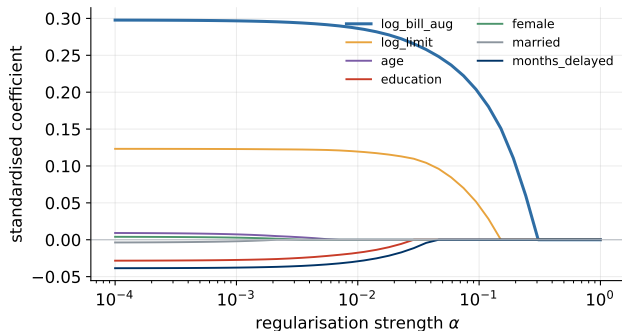
- As α grows, every coefficient shrinks toward zero - smoothly, **never exactly to zero**.
- Loves correlated features: splits credit between twins instead of gambling on one. The multicollinearity remedy.



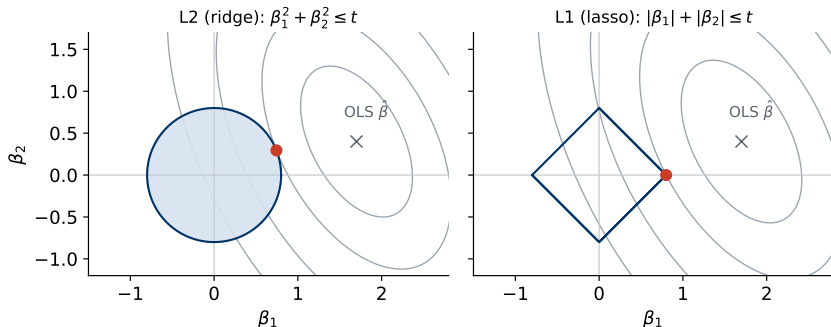
Lasso (L1): shrinkage with a guillotine

$$\min_{\beta} \sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_j |\beta_j|$$

- Coefficients hit **exactly zero** one by one - built-in feature selection; the survivors are your shortlist.
- With twin features it tends to keep one and drop the other - convenient, but don't read the choice causally.
- Mind the α axis: L1 bites at much smaller α than ridge did.



Why the guillotine: the geometry of L1 vs L2



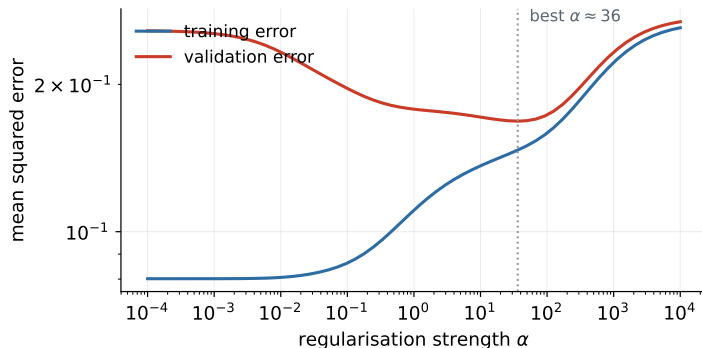
- Grey ellipses: equal-loss curves around the unpenalised OLS solution. Shaded region: the **coefficient budget** the penalty allows. The solution is the first touch.
- A circle can be touched anywhere $\Rightarrow$ ridge coefficients get *small*, not zero. A diamond gets hit at its **corners** - and corners have $\beta_j = 0$ exactly.

Scale first, or the penalty is nonsense

- The penalty charges by **coefficient size** - and size depends on units. Limit-in-NT\$ gets a tiny coefficient, age-in-years a big one, for identical importance.
- Unscaled features $\Rightarrow$ the penalty punishes features for their *units*, not their usefulness.
- Standard fix: `StandardScaler` (mean 0, SD 1) before any penalised model.
- And the Block 1 golden rule again: the scaler's mean and SD are **fitted on train only** - test data must not leak into them.

“But sklearn didn't warn me” - correct, it won't. This one is on you.

Choosing α : validation, not vibes



- α is a **hyperparameter**: the data used to fit coefficients cannot also pick it.
- Sweep α , judge on held-out data, take the bottom of the U. Done properly (cross-validation) next week.

Regularisation: practical notes

- **Elastic net** = ridge + lasso mixed; when in doubt between them, mix.
- **Gotcha**: in `LogisticRegression` the knob is $C = 1/\alpha$ - **bigger C = weaker penalty** - and regularisation is **on by default** ($C=1$). You have been regularising all along without knowing.
- Ridge/lasso logic transfers to logistic regression unchanged (penalise the same β 's under log-loss).
- Regularisation controls *coefficients*, not data quality: it will happily shrink its way around leakage without curing it.

Regularisation is everywhere (a preview)

Penalties are one instance of a universal idea: **put a leash on flexibility, and generalisation improves.**

Leash	Model family	Where in this course
L1 / L2 penalty	linear & logistic	today
Max depth, min leaf size	decision trees	Block 3
Fewer boosting rounds, learning rate	GBDT	Block 3
Early stopping, dropout	neural networks	Block 6
Coarse bins (WoE)	scorecards	Blocks 1–2

- Different mechanics, one U-curve: each knob trades training fit for test stability.
- When you meet a new model family, your first question is now fixed: **“where is its leash?”**

The leakage-safe pattern: Pipeline

Weld the preprocessing to the model:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(StandardScaler(),
                     LogisticRegression(max_iter=2000))
pipe.fit(X_train, y_train)          # scaler fitted on train ONLY
proba = pipe.predict_proba(X_test)[: , 1]
```

- The pipeline welds preprocessing to the model: whatever data the model sees, the transforms were fitted on *its training part*.
- This is not style advice - next week, inside cross-validation, pipelines are the **only** way to stay leakage-free.

The accuracy trap, now with our own numbers

Our PD baseline on the held-out test set (9,000 real clients):

Model	Accuracy
Always predict “repaid”	77.9%
Our logistic baseline (threshold 0.5)	80.6%

- Three points better than a model that *does nothing*. Is our model useless?
- No - it **rank**s customers well; accuracy at an arbitrary threshold just cannot see it.
- Diagnosis requires opening the black box of “right and wrong” - next slide.

The confusion matrix

		Predicted	
		default	repaid
Actual	default	TP = 541 caught	FN = 1,450 missed
	repaid	FP = 294 falsely declined	TN = 6,715 rightly approved

- Four numbers, four different business events - **never** let them collapse into one “accuracy” without looking.
- Our baseline at 0.5: cautious - it declines rarely (294 false alarms) but **misses 1,450 of 1,991 defaulters**.

Errors have price tags - different ones

False negative (missed defaulter)

- loan granted, money lost
- cost $\sim$ the **principal**
- The two costs differ by an **order of magnitude** - and the ratio belongs to the business, not to the data scientist.
- A model tuned for symmetric errors is tuned for a fictional bank.
- This asymmetry, not mathematics, will pick our threshold.

False positive (good customer declined)

- margin lost, customer possibly gone for good
- cost $\sim$ the *margin*

Precision and recall

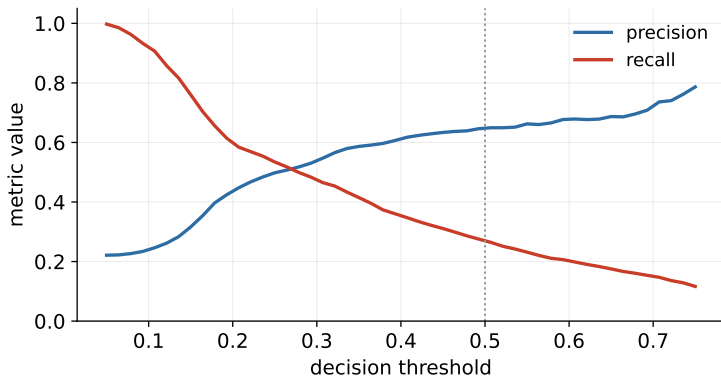
$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

- **Precision:** when we flag a defaulter, how often are we right? Ours: $541/835 \approx 65\%$ - against a 22% base rate, that is real skill.
- **Recall:** of all true defaulters, how many do we catch? Ours: $541/1991 \approx 27\%$ - nearly three out of four walk through.
- The pair always trades off - and the exchange rate is the threshold.

(F1 = harmonic mean of the two - one number, both flavours, and blind to costs and true negatives.
Use knowingly or not at all.)

Move the threshold, move the business



- Lower the threshold: catch more defaulters (recall $\uparrow$), decline more good customers (precision $\downarrow$).
- The threshold is a **business decision** expressed in code - risk appetite, not arithmetic.

Which metric for PD, then?

- **Accuracy**: no. (You have seen why, twice.)
- **Precision/recall**: honest but **threshold-bound** - they judge one cut-off, not the model.
- What we actually want to know, in order:
 - 1 Does the model *rank* risk well? → ROC / AUC (next week)
 - 2 Are the probabilities *honest*? → calibration (next week)
 - 3 What does the best threshold *earn*? → expected-cost analysis (next week, with your numbers)
- Until then, one rule survives every situation: **never a single metric, never on training data.**

Laboratory

Notebook 02_regression_baselines - laptops out.

Lab today

Notebook 02_regression_baselines

- split the portfolio (stratified, seeded)
- linear warm-up: payments from bills
- logistic PD baseline in a pipeline
- read coefficients & odds ratios
- play the threshold game

Done =

- you can defend your baseline: features used, one number you trust, and *why* that number
- you know your model's confusion matrix at two thresholds

Summary

What to take home from Block 2.

Block 2 in five sentences

- 1 Judge models only on **unseen data** - the train/test split is the minimum honest protocol.
- 2 Linear regression predicts numbers; its coefficients are **unit-bound teammates**, and residual plots tell the truth.
- 3 Logistic regression is linear in **log-odds**; e^β multiplies odds, which is exactly how credit talks.
- 4 **Regularisation** (ridge smoothly, lasso to zero) buys test error with bias - after scaling, chosen by validation.
- 5 Accuracy lies under imbalance; read the **confusion matrix**, price its cells, and treat the **threshold** as a business decision.

Project status & reminder

- Teams of 3 closed **today at 17:10** - not in one? Talk to me in the break, we fix it now.
- **Before Block 3 (20.10): topic + short written description** - the decision, the dataset, the target, the planned two model families. No idea? **Consult before the deadline.**
- One of your two families should probably be today's logistic baseline - cheap, defensible, expected.

Rubric reminder: "Modelling: ≥ 2 families, justified" is 15% - and "justified" means *against a baseline*.

Exam practice - multiple choice

Q1. A standardised logistic coefficient for `months_delayed_6m` is $+0.6$. The correct reading:

- ☐ a) default probability rises by 0.6 pp per month of delay
- ☐ b) one SD more delay adds 0.6 to the log-odds - $\text{odds} \times e^{0.6} \approx 1.8$
- ☐ c) default probability multiplies by 1.8 for every customer
- ☐ d) the feature explains 60% of defaults

Q2. Why must the scaler live *inside* the pipeline, fitted on training data only?

- ☐ a) it makes the code shorter
- ☐ b) fitting it on all rows lets test-set statistics leak into training
- ☐ c) scaling the test set is forbidden
- ☐ d) scikit-learn refuses to fit it otherwise

Exam practice - open question

In the style of the term-0 exam (~60% open questions)

At threshold 0.5 your PD baseline catches 27% of defaulters at 65% precision; at threshold 0.30 it catches far more defaulters at visibly lower precision. A manager asks: “which threshold is correct?” Answer in 4-6 sentences.

Hint: the best answers contain no model mathematics at all.

Next week: trees, ensembles & honest
evaluation.

The `collections_contact` trap finally springs - live.

Keep learning - Block 2 reading

- **Shmueli** - *To Explain or to Predict?* (Statistical Science, 2010) - the inference-vs-prediction re-framing, in one readable paper.
- **Tibshirani** - *Regression Shrinkage and Selection via the Lasso* (JRSS-B, 1996) - the original lasso paper; the geometry slide, from the source.
- **Hand & Henley** - *Statistical Classification Methods in Consumer Credit Scoring: a Review* (JRSS-A, 1997) - the scorecard tradition our odds-ratio talk descends from.
- **scikit-learn User Guide** - *Linear models* section: regularisation explained better than most textbooks.